

```
from google.colab import files
upload =files.upload()
```

[Show hidden output](#)

```
from google.colab import files
upload =files.upload()
```

No file chosen
cell to enable.
Saving train.csv to train.csv

Upload widget is only available when the cell has been executed in the current browser session. Please rerun this



Project Description

This project focuses on building a Demand Forecasting system for retail stores using machine learning techniques. The main objective is to predict daily sales for each store based on historical data and various influencing factors such as promotions, holidays, competition, and customer behavior.

The project simulates a real-world business problem where accurate sales predictions can help companies optimize inventory, improve marketing strategies, and make better operational decisions. It includes a complete pipeline starting from data exploration and preprocessing to model training, evaluation, and deployment.

Data Description

The dataset contains daily sales records for Rossmann stores in Germany. Each row represents a store's performance on a specific day, including features such as store ID, date, sales (target), customers, promotions, holidays, store type, and competition-related information.

Column-by-column description

1.Store

- Unique identifier for each store branch.

2.DayOfWeek

- Represents the day of the week (from 1 to 7) on which the sales were recorded, and is used to analyze weekly sales patterns.

3.Date

- The specific date of the recorded sales.

4.Sales

- Total daily revenue (turnover) generated by the store (Target variable).

5. Customers

- Number of customers who made purchases in the store on that day.

6. Open

- Indicates whether the store was open (1) or closed (0) on that day.

7. StateHoliday

- Indicates if the day was a public/state holiday
- (a = public holiday, b = Easter, c = Christmas, 0 = none).

8. SchoolHoliday

- Indicates whether public schools were closed on that day.

9. StoreType

- Categorizes the store into one of four different store models (a, b, c, d).
- (A = Hypermarket & B = Small Store & C = Specialty & D = Premium)

10. Assortment

- Describes the product assortment level
- (a = basic, b = extra, c = extended).

11. CompetitionDistance

- Distance (in meters) to the nearest competitor store.

12. CompetitionOpenSinceMonth

- The month when the nearest competitor store opened.

13. CompetitionOpenSinceYear

- The year when the nearest competitor store opened.

14. Promo

- Indicates whether the store was running a promotion on that day (1 = yes, 0 = no).

15. Promo2

- Indicates whether the store is participating in a continuous promotion program (1 = yes, 0 = no).

16. Promo2SinceYear

- The year when the store started participating in Promo2.

17. Promo2SinceWeek

- The calendar week when the store started participating in Promo2.

18. PromoInterval

- Describes the months when Promo2 is activated each year

Goal

The main goal of this project is to accurately predict daily sales for retail stores using machine learning models.

This helps businesses to:

- Optimize inventory management
- Improve promotional strategies
- Reduce losses and overstocking
- Make data-driven decisions

In short, the project aims to transform raw retail data into actionable insights and reliable sales forecasts.

✓ The Code

```
# load data from two files
import pandas as pd
train_data = pd.read_csv("train.csv")
store_data = pd.read_csv("store.csv")

# Merge train_data and store_data
original_df = train_data.merge(store_data, on="Store", how="left")

# check
original_df.head()
```

```
/tmp/ipykernel_16010/1167660996.py:3: DtypeWarning: Columns (7) have mixed types. Specify dtype option on import or set low_memory=True
train_data = pd.read_csv("train.csv")
```

	Store	DayOfWeek	Date	Sales	Customers	Open	Promo	StateHoliday	SchoolHoliday	StoreType	Assortment	CompetitionDistance
0	1	5	2015-07-31	5263	555	1	1	0	1	c	a	1270.0
1	2	5	2015-07-31	6064	625	1	1	0	1	a	a	570.0
2	3	5	2015-07-31	8314	821	1	1	0	1	a	a	14130.0
3	4	5	2015-07-31	13995	1498	1	1	0	1	c	c	620.0
4	5	5	2015-07-31	4822	559	1	1	0	1	a	a	29910.0

```
# copy dataframe
df = original_df.copy()
```

```
# Library use

import numpy as np
# Importing NumPy for numerical operations (arrays, math functions, etc.)
import pandas as pd
# Importing Pandas for data manipulation and analysis (DataFrames, CSV files, etc.)
import matplotlib.pyplot as plt
# Importing Matplotlib's Pyplot for basic plotting and data visualization
import seaborn as sns
import plotly.express as px
# Importing Seaborn,plotly for advanced statistical data visualization (built on top of Matplotlib)

# Sklearn
from sklearn.model_selection import train_test_split
# Splits the dataset into training and testing sets to evaluate the model properly.
from sklearn.preprocessing import StandardScaler, OneHotEncoder, LabelEncoder, MinMaxScaler
# StandardScaler => Standardizes numeric features to mean = 0 and std = 1.
# OneHotEncoder => Converts categorical columns into multiple binary (0/1) columns.
# LabelEncoder => Converts categorical labels into integer values (0, 1, 2,...).
# MinMaxScaler => Scales values to a fixed range, usually between 0 and 1.

from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
```

⌵ Exploratory Data Analysis (EDA)

```
# Display Data info
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1017209 entries, 0 to 1017208
Data columns (total 18 columns):
 #   Column                Non-Null Count  Dtype
---  ---
 0   Store                 1017209 non-null  int64
 1   DayOfWeek             1017209 non-null  int64
 2   Date                  1017209 non-null  object
 3   Sales                 1017209 non-null  int64
 4   Customers             1017209 non-null  int64
```

```
5 Open 1017209 non-null int64
6 Promo 1017209 non-null int64
7 StateHoliday 1017209 non-null object
8 SchoolHoliday 1017209 non-null int64
9 StoreType 1017209 non-null object
10 Assortment 1017209 non-null object
11 CompetitionDistance 1014567 non-null float64
12 CompetitionOpenSinceMonth 693861 non-null float64
13 CompetitionOpenSinceYear 693861 non-null float64
14 Promo2 1017209 non-null int64
15 Promo2SinceWeek 509178 non-null float64
16 Promo2SinceYear 509178 non-null float64
17 PromoInterval 509178 non-null object
dtypes: float64(5), int64(8), object(5)
memory usage: 139.7+ MB
```

```
# generate quick statistical summaries for the numeric columns
df.describe()
```

	Store	DayOfWeek	Sales	Customers	Open	Promo	SchoolHoliday	CompetitionDistance	CompetitionOpenSinceMonth
count	1.017209e+06	1.017209e+06	1.017209e+06	1.017209e+06	1.017209e+06	1.017209e+06	1.017209e+06	1.014567e+06	6.93861e+05
mean	5.584297e+02	3.998341e+00	5.773819e+03	6.331459e+02	8.301067e-01	3.815145e-01	1.786467e-01	5.430086e+03	1.017209e+06
std	3.219087e+02	1.997391e+00	3.849926e+03	4.644117e+02	3.755392e-01	4.857586e-01	3.830564e-01	7.715324e+03	1.017209e+06
min	1.000000e+00	1.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00	2.000000e+01	1.017209e+06
25%	2.800000e+02	2.000000e+00	3.727000e+03	4.050000e+02	1.000000e+00	0.000000e+00	0.000000e+00	7.100000e+02	1.017209e+06
50%	5.580000e+02	4.000000e+00	5.744000e+03	6.090000e+02	1.000000e+00	0.000000e+00	0.000000e+00	2.330000e+03	1.017209e+06
75%	8.380000e+02	6.000000e+00	7.856000e+03	8.370000e+02	1.000000e+00	1.000000e+00	0.000000e+00	6.890000e+03	1.017209e+06
max	1.115000e+03	7.000000e+00	4.155100e+04	7.388000e+03	1.000000e+00	1.000000e+00	1.000000e+00	7.586000e+04	1.017209e+06

```
# generates statistical summaries for object (categorical/text) columns
df.describe(include="object")
```

	Date	StateHoliday	StoreType	Assortment	PromoInterval
count	1017209	1017209	1017209	1017209	509178
unique	942	5	4	3	3
top	2013-01-02	0	a	a	Jan, Apr, Jul, Oct
freq	1115	855087	551627	537445	293122

```
# code to separate the numerical columns and categorical columns
```

```
num_cols = [col for col in df.columns if df[col].dtype != "object"]
cat_cols = [col for col in df.columns if df[col].dtype == "object"]
```

```
print("Numerical Columns:", num_cols)
print("Categorical Columns:", cat_cols)
```

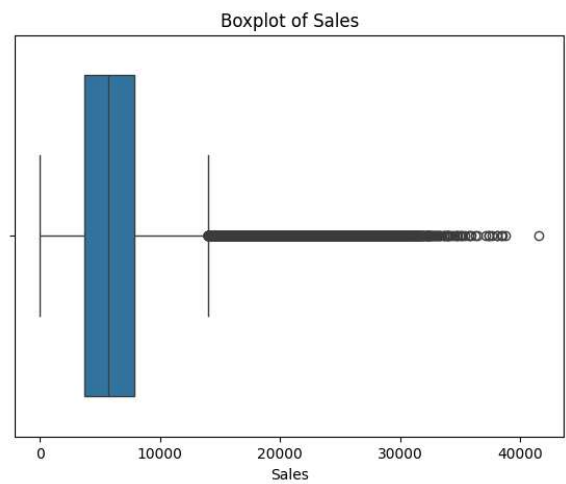
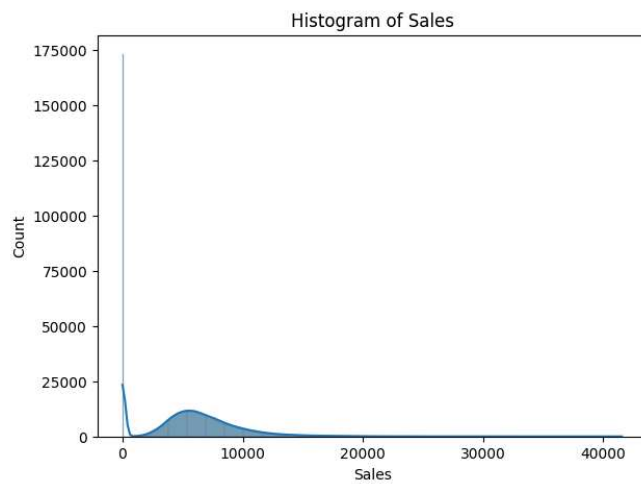
```
Numerical Columns: ['Store', 'DayOfWeek', 'Sales', 'Customers', 'Open', 'Promo', 'SchoolHoliday', 'CompetitionDistance', 'CompetitionOpenSinceMonth', 'CompetitionOpenSinceYear']
Categorical Columns: ['Date', 'StateHoliday', 'StoreType', 'Assortment', 'PromoInterval']
```

Visualization

```
# draw histogram and boxplot for the column (sales)
```

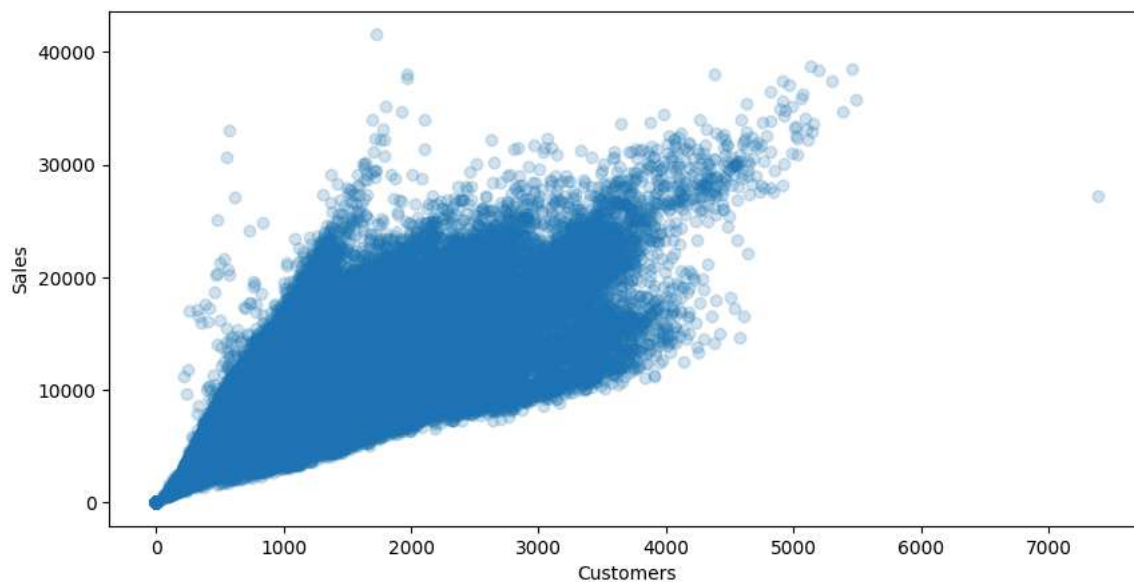
```
# histogram
fig, ax = plt.subplots(1, 2, figsize = (15, 5))
sns.histplot(x = df["Sales"], kde = True, ax = ax[0])
ax[0].set_title("Histogram of Sales")
```

```
# boxplot
sns.boxplot(x = df["Sales"], ax = ax[1])
ax[1].set_title("Boxplot of Sales")
plt.show()
```



```
# show the relationship between the customers and the sales
```

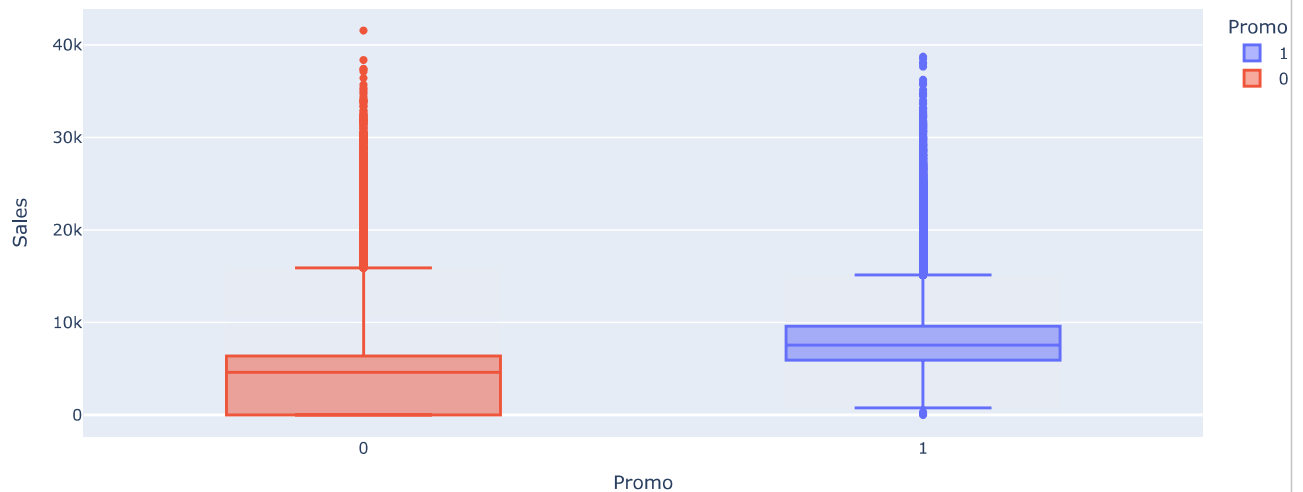
```
plt.figure(figsize = (10 ,5))
plt.scatter(x = df['Customers'] , y = df["Sales"] , alpha = 0.2 )
plt.xlabel("Customers")
plt.ylabel("Sales")
plt.show()
```



```
# display the relationship between the sales and the Promo using plotly
```

```
fig = px.box(df , x = 'Promo' , y = 'Sales' , title = 'Relationship between Sales and Promo' , width= 1000 , height= 500 ,color
fig.show()
```

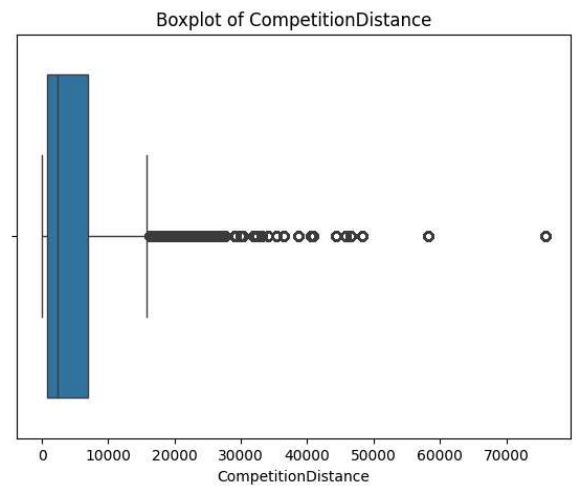
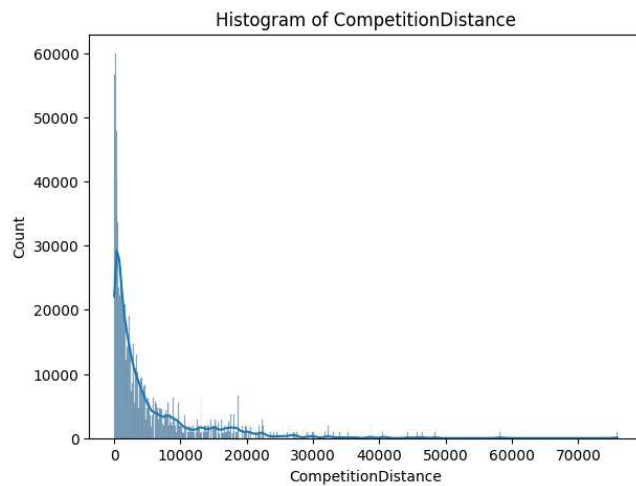
Relationship between Sales and Promo



```
# draw histogram and boxplot for the column (CompetitionDistance)

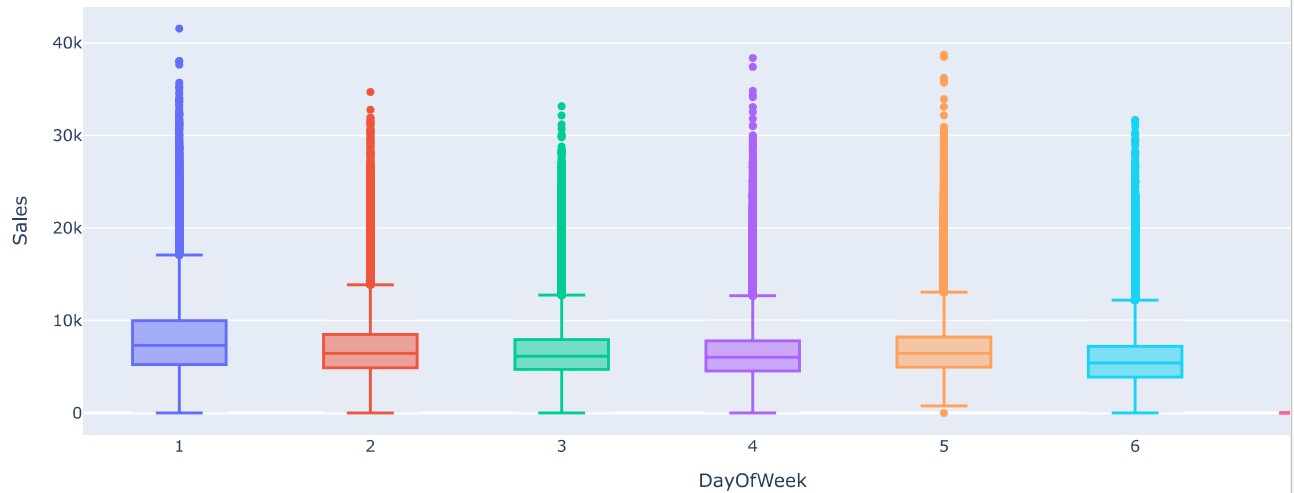
# histogram
fig , ax = plt.subplots(1 ,2 ,figsize = (15 ,5))
sns.histplot(x = df["CompetitionDistance"] , kde = True ,ax = ax[0])
ax[0].set_title ("Histogram of CompetitionDistance")

# boxplot
sns.boxplot(x = df["CompetitionDistance"] , ax = ax[1])
ax[1].set_title ("Boxplot of CompetitionDistance")
plt.show()
```



```
# show the relationship between DayOfWeek & Sales
# Notes => Monday(1) to Sunday(7)
plt.figure(figsize=(10,5))
fig = px.box(df, x='DayOfWeek', y='Sales', title=' DayOfWeek & Sales', width=1200, height=500 ,color='DayOfWeek' ,category_orde
fig.show()
```

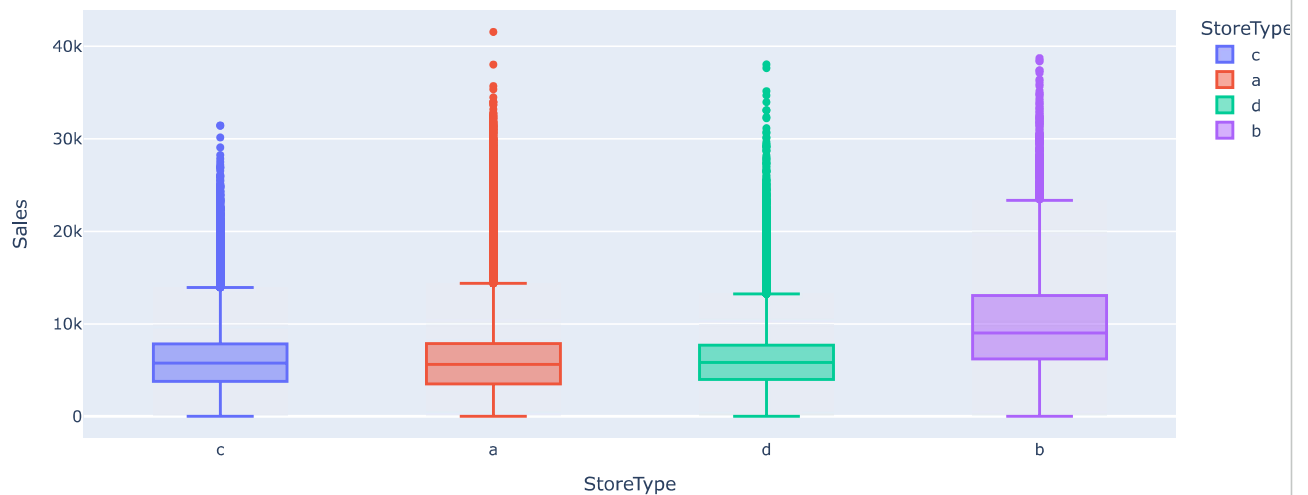
DayOfWeek & Sales



<Figure size 1000x500 with 0 Axes>

```
# show the relationship between StoreType & Sales
plt.figure(figsize=(10,5))
fig = px.box( df ,x='StoreType', y='Sales', title=' StoreType & Sales', width=1000, height=500 ,color='StoreType')
fig.show()
```

StoreType & Sales



<Figure size 1000x500 with 0 Axes>

EDA Notes

- The distribution of Sales is right-skewed, with several high-value outliers.
- There is a strong positive relationship between Customers and Sales, indicating that higher customer counts lead to higher sales.
- Promotional campaigns (Promo) have a significant impact on increasing sales compared to non-promotion days.
- Sales vary across days of the week, suggesting the presence of weekly seasonality.
- CompetitionDistance is highly skewed, with most competitors located relatively close while a few are very far away.
- Store type (StoreType) influences sales performance, with some store types achieving higher median sales than others.
- Several columns contain missing values, which appear to represent logical absence rather than data errors.

- Categorical variables such as StoreType, Assortment, and StateHoliday will require encoding .

▼ **Dtat proccessing pipeline**

1. Genralization
2. Handling Missing Values
3. Feature Engineering
4. Encoding
5. Handling Outliers
6. Feature Selection
7. Train-Test Split

1. Genralization

```
# Check  
df.head()
```

	Store	DayOfWeek	Date	Sales	Customers	Open	Promo	StateHoliday	SchoolHoliday	StoreType	Assortment	CompetitionDistance
0	1	5	2015-07-31	5263	555	1	1	0	1	c	a	1270.0
1	2	5	2015-07-31	6064	625	1	1	0	1	a	a	570.0
2	3	5	2015-07-31	8314	821	1	1	0	1	a	a	14130.0
3	4	5	2015-07-31	13995	1498	1	1	0	1	c	c	620.0
4	5	5	2015-07-31	4822	559	1	1	0	1	a	a	29910.0

```
# show types of columns  
df.dtypes
```

	0
Store	int64
DayOfWeek	int64
Date	object
Sales	int64
Customers	int64
Open	int64
Promo	int64
StateHoliday	object
SchoolHoliday	int64
StoreType	object
Assortment	object
CompetitionDistance	float64
CompetitionOpenSinceMonth	float64
CompetitionOpenSinceYear	float64
Promo2	int64
Promo2SinceWeek	float64
Promo2SinceYear	float64
PromoInterval	object

dtype: object

```
# code for remove right and left spaces and convert catigrical columns into capitalize
for col in cat_cols:
    df[col] = df[col].str.strip().str.capitalize()
```

```
# convert data to datetime
df["Date"] = pd.to_datetime(df["Date"])
```

```
# show the unique values of column (StateHoliday)
df["StateHoliday"].unique()
```

```
array(['0', 'A', 'B', 'C', nan], dtype=object)
```

```
# Notes => nan usually mean no holiday so we will fill it with 0 too,
df["StateHoliday"] = df["StateHoliday"].fillna(0)
# sometime read 0 as string so we convert it to int
df["StateHoliday"] = df["StateHoliday"].replace("0", 0)
```

```
# check if there is duplicated rows
df.duplicated().sum()
```

```
np.int64(0)
```

```
# Check the unique values of each column
df.nunique()
```

	0
Store	1115
DayOfWeek	7
Date	942
Sales	21734
Customers	4086
Open	2
Promo	2
StateHoliday	4
SchoolHoliday	2
StoreType	4
Assortment	3
CompetitionDistance	654
CompetitionOpenSinceMonth	12
CompetitionOpenSinceYear	23
Promo2	2
Promo2SinceWeek	24
Promo2SinceYear	7
PromoInterval	3

dtype: int64

2. Handling Missing Values

```
# Check missing values
df.isnull().sum()
```

	0
Store	0
DayOfWeek	0
Date	0
Sales	0
Customers	0
Open	0
Promo	0
StateHoliday	0
SchoolHoliday	0
StoreType	0
Assortment	0
CompetitionDistance	2642
CompetitionOpenSinceMonth	323348
CompetitionOpenSinceYear	323348
Promo2	0
Promo2SinceWeek	508031
Promo2SinceYear	508031
PromoInterval	508031

dtype: int64

```
# fill missing value in CompetitionDistance with median
df["CompetitionDistance"] = df["CompetitionDistance"].fillna(df["CompetitionDistance"].median())
```

```
# fill missing values in CompetitionOpenSinceMonth and CompetitionOpenSinceYear with 0
# Notes => 0 represent to unknown value
df["CompetitionOpenSinceMonth"] = df["CompetitionOpenSinceMonth"].fillna(0)
df["CompetitionOpenSinceYear"] = df["CompetitionOpenSinceYear"].fillna(0)
```

```
# fill missing values with 0 for num_cols and None for cat_cols
df["Promo2SinceWeek"] = df["Promo2SinceWeek"].fillna(0)
df["Promo2SinceYear"] = df["Promo2SinceYear"].fillna(0)
df["PromoInterval"] = df["PromoInterval"].fillna("None")
```

```
# Check
df.isnull().sum()
```

	0
Store	0
DayOfWeek	0
Date	0
Sales	0
Customers	0
Open	0
Promo	0
StateHoliday	0
SchoolHoliday	0
StoreType	0
Assortment	0
CompetitionDistance	0
CompetitionOpenSinceMonth	0
CompetitionOpenSinceYear	0
Promo2	0
Promo2SinceWeek	0
Promo2SinceYear	0
PromoInterval	0

dtype: int64

3. Feature Engineering

```
df.head()
```

	Store	DayOfWeek	Date	Sales	Customers	Open	Promo	StateHoliday	SchoolHoliday	StoreType	Assortment	CompetitionDistance
0	1	5	2015-07-31	5263	555	1	1	0	1	C	A	1270.0
1	2	5	2015-07-31	6064	625	1	1	0	1	A	A	570.0
2	3	5	2015-07-31	8314	821	1	1	0	1	A	A	14130.0
3	4	5	2015-07-31	13995	1498	1	1	0	1	C	C	620.0
4	5	5	2015-07-31	4822	559	1	1	0	1	A	A	29910.0

```
# Extract some useful features from column Data
```

```
df["Year"] = df["Date"].dt.year
df["Month"] = df["Date"].dt.month
df["DayOfMonth"] = df["Date"].dt.day

# column to check if is start and end of the month
df["startmonth"] = df['DayOfMonth'].isin([1,2,3,4,5,6,7]).astype(int)
df["endmonth"] = df['DayOfMonth'].isin([25,26,27,28,29,30,31]).astype(int)

# column to check if Is Weekend
df["IsWeekend"] = df["DayOfWeek"].isin([5,6]).astype(int)
```

```
# create column to check if CompetitionDate is Known or not
df["CompetitionDateKnown"] = (df["CompetitionOpenSinceYear"] != 0).astype(int)
```

```
# Detrmine the competition duration (if the data is known)
# Note => errors if found any NaN or 0 pandas convert it to Nat which mean Not a Time
df["CompetitionOpenDate"] = pd.to_datetime(
    dict(
        year = df["CompetitionOpenSinceYear"],
        month = df["CompetitionOpenSinceMonth"],
        day = 1
    ),
    errors = "coerce"
)
```

```
# create column detrmine the competition duration and convert it to months
df["CompetitionDurationMonths"] = (df['Date'] - df['CompetitionOpenDate']).dt.days/30
```

```
# handle missing in CompetitionDurationMonths
df["CompetitionDurationMonths"] = df["CompetitionDurationMonths"].fillna(0)

# Notes => Sometimes CompetitionOpenDate > Date ,so we found negative values
# filter to convert any negative value to 0
df["CompetitionDurationMonths"] = df["CompetitionDurationMonths"].clip(lower=0)
```

```
df["IsPromo"] = df["Promo"]
df["IsPromo2"] = df["Promo2"]
```

```
df.columns
```

```
Index(['Store', 'DayOfWeek', 'Date', 'Sales', 'Customers', 'Open', 'Promo',
      'StateHoliday', 'SchoolHoliday', 'StoreType', 'Assortment',
      'CompetitionDistance', 'CompetitionOpenSinceMonth',
      'CompetitionOpenSinceYear', 'Promo2', 'Promo2SinceWeek',
      'Promo2SinceYear', 'PromoInterval', 'Year', 'Month', 'DayOfMonth',
      'startmonth', 'endmonth', 'IsWeekend', 'CompetitionDateKnown',
      'CompetitionOpenDate', 'CompetitionDurationMonths', 'IsPromo',
      'IsPromo2'],
      dtype='object')
```

```
# Drop unimportant columns
df.drop(["Store", "Promo", "Promo2", "CompetitionOpenDate", "Date", "Promo2SinceWeek",
        "Promo2SinceYear", "PromoInterval", "DayOfMonth", "CompetitionOpenSinceMonth", "CompetitionOpenSinceYear"], axis=1, inplace=True)
```

```
# filter the rows that open = 1
df = df[df["Open"] == 1]
```

```
# drop Open
df.drop("Open", axis=1, inplace=True)
```

```
df.head()
```

	DayOfWeek	Sales	Customers	StateHoliday	SchoolHoliday	StoreType	Assortment	CompetitionDistance	Year	Month	startmonth
0	5	5263	555	0	1	C	A	1270.0	2015	7	0
1	5	6064	625	0	1	A	A	570.0	2015	7	0
2	5	8314	821	0	1	A	A	14130.0	2015	7	0
3	5	13995	1498	0	1	C	C	620.0	2015	7	0
4	5	4822	559	0	1	A	A	29910.0	2015	7	0

```
# Check missing values after feature engineering
df.isnull().sum()
```

```

0
DayOfWeek      0
Sales          0
Customers      0
StateHoliday   0
SchoolHoliday  0
StoreType      0
Assortment     0
CompetitionDistance  0
Year          0
Month         0
startmonth     0
endmonth       0
IsWeekend     0
CompetitionDateKnown  0
CompetitionDurationMonths  0
IsPromo       0
IsPromo2      0

```

```
dtype: int64
```

4. Encoding

```
# onehotencoding for StoreType
# object
ohe_StoreType = OneHotEncoder(sparse_output=False)
# ohe return array
Storetype_array = ohe_StoreType.fit_transform(df[["StoreType"]])
# convert array to dataframe
df_Storetype = pd.DataFrame(Storetype_array , columns=ohe_StoreType.get_feature_names_out(["StoreType"]))
# Combine the df_Storetype with df
pd.concat([df , df_Storetype] , axis=1)
# Drop the column StoreType
df.drop("StoreType", axis=1, inplace=True)
```

```
# onehotencoding for Assortment
# object
ohe_Assortment = OneHotEncoder(sparse_output=False)
# ohe return array
Assortment_array = ohe_Assortment.fit_transform(df[["Assortment"]])
# convert array to dataframe
df_Assortment = pd.DataFrame(Assortment_array , columns=ohe_Assortment.get_feature_names_out(["Assortment"]))
# Combine the df_Storetype with df
pd.concat([df , df_Assortment] , axis=1)
# Drop the column StoreType
df.drop("Assortment", axis=1, inplace=True)
```

```
# convert to string
df["StateHoliday"] = df["StateHoliday"].astype(str)

# onehotencoding for StateHoliday
# object
ohe_StateHoliday = OneHotEncoder(sparse_output=False)
# ohe return array
StateHoliday_array = ohe_StateHoliday.fit_transform(df[["StateHoliday"]])
# convert array to dataframe
df_StateHoliday = pd.DataFrame(StateHoliday_array , columns=ohe_StateHoliday.get_feature_names_out(["StateHoliday"]))
# Combine the df_Storetype with df
pd.concat([df , df_StateHoliday] , axis=1)
# Drop the column StoreType
df.drop("StateHoliday", axis=1, inplace=True)
```

```
# Check
df.head()
```

	DayOfWeek	Sales	Customers	SchoolHoliday	CompetitionDistance	Year	Month	startmonth	endmonth	IsWeekend	CompetitionDate
0	5	5263	555	1	1270.0	2015	7	0	1	1	
1	5	6064	625	1	570.0	2015	7	0	1	1	
2	5	8314	821	1	14130.0	2015	7	0	1	1	
3	5	13995	1498	1	620.0	2015	7	0	1	1	
4	5	4822	559	1	29910.0	2015	7	0	1	1	

5. Handling Outliers

```
# List continue continous columns
continous_cols = ["Sales" , "CompetitionDistance" , "CompetitionDurationMonths"] # Customers
```

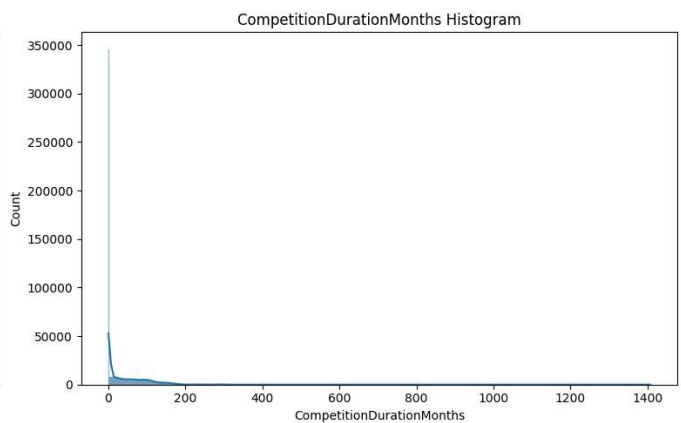
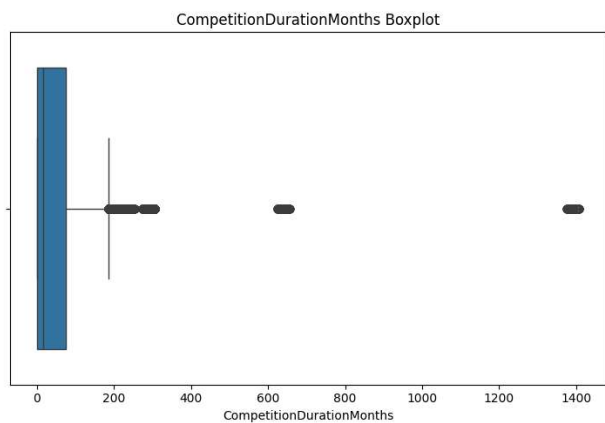
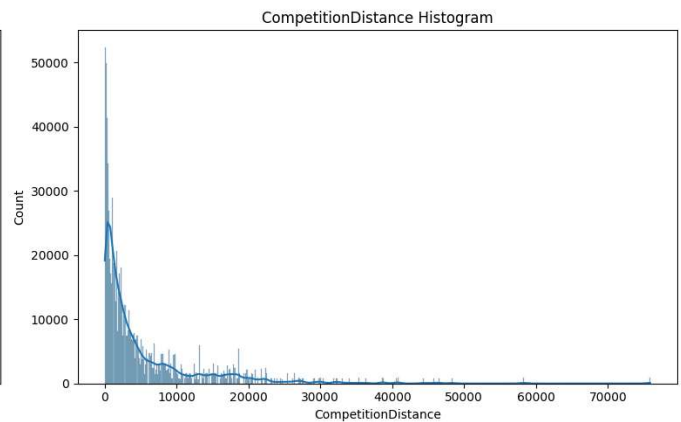
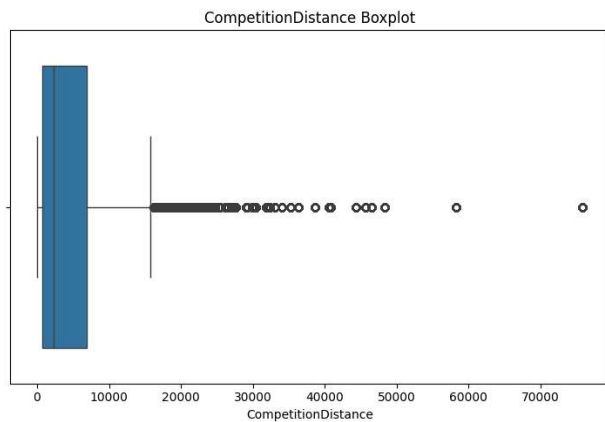
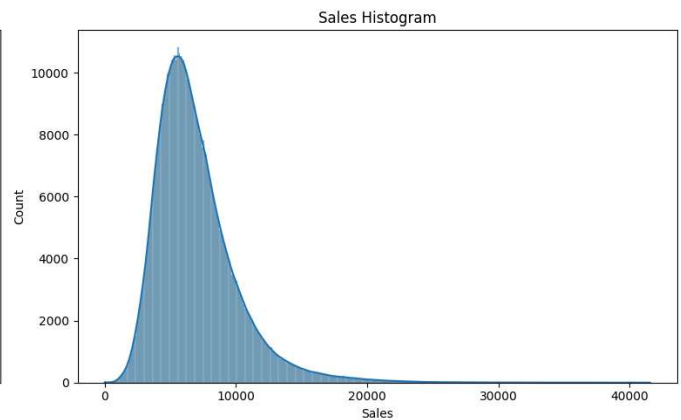
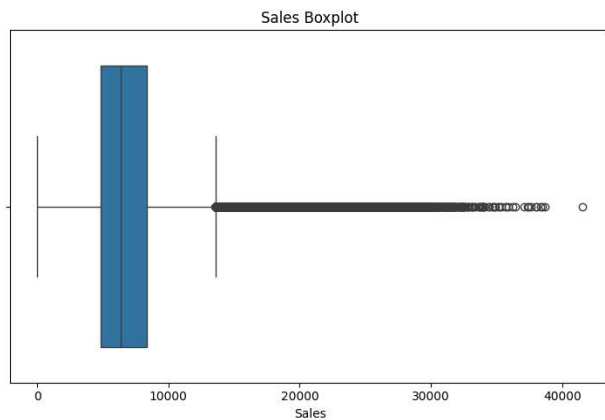
```
#Code draw the Boxplot and histogram for each column to show outliers

for col in continous_cols:
    fig, ax = plt.subplots(1, 2, figsize=(15, 5))

    # Boxplot
    ax[0].set_title(f'{col} Boxplot')
    sns.boxplot(x=df[col], ax=ax[0])

    # Histogram
    ax[1].set_title(f'{col} Histogram')
    sns.histplot(x=df[col], ax=ax[1], kde=True)

# tight_layout() => Adjusts spacing automatically to avoid overlap between titles and axes.
plt.tight_layout()
plt.show()
```



```
# Use Log Transformation to delete Outliers
df["Sales_log"] = np.log1p(df["Sales"])
df['CompetitionDistance_log'] = np.log1p(df['CompetitionDistance'])
df['CompetitionDurationMonths_log'] = np.log1p(df['CompetitionDurationMonths'])
df["Customers"] = np.log1p(df["Customers"])
```

```
df = df.drop(columns=[
    "Sales",
    "CompetitionDistance",
    "CompetitionDurationMonths",
])
```

```
df.head()
```

	DayOfWeek	Customers	SchoolHoliday	Year	Month	startmonth	endmonth	IsWeekend	CompetitionDateKnown	IsPromo	IsPromo2	Sa
0	5	6.320768	1	2015	7	0	1	1	1	1	0	8
1	5	6.439350	1	2015	7	0	1	1	1	1	1	8
2	5	6.711740	1	2015	7	0	1	1	1	1	1	9
3	5	7.312553	1	2015	7	0	1	1	1	1	0	9
4	5	6.327937	1	2015	7	0	1	1	1	1	0	8

```

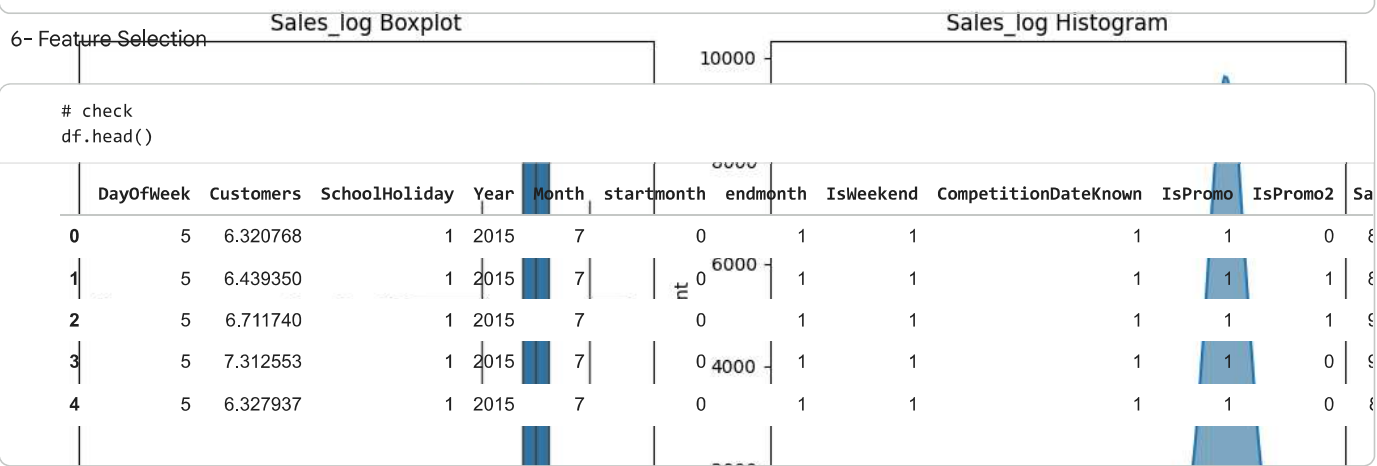
for col in ["Sales_log", "CompetitionDistance_log", "CompetitionDurationMonths_log"]:
    fig, ax = plt.subplots(1, 2, figsize=(10, 5))

    # Boxplot
    ax[0].set_title(f'{col} Boxplot')
    sns.boxplot(x=df[col], ax=ax[0])

    # Histogram
    ax[1].set_title(f'{col} Histogram')
    sns.histplot(x=df[col], ax=ax[1], kde=True)

    # tight_layout() => Adjusts spacing automatically to avoid overlap between titles and axes.
    plt.tight_layout()
    plt.show()

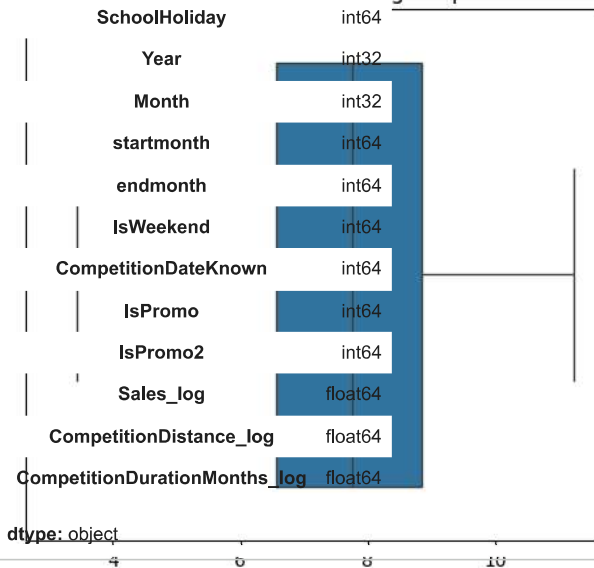
```

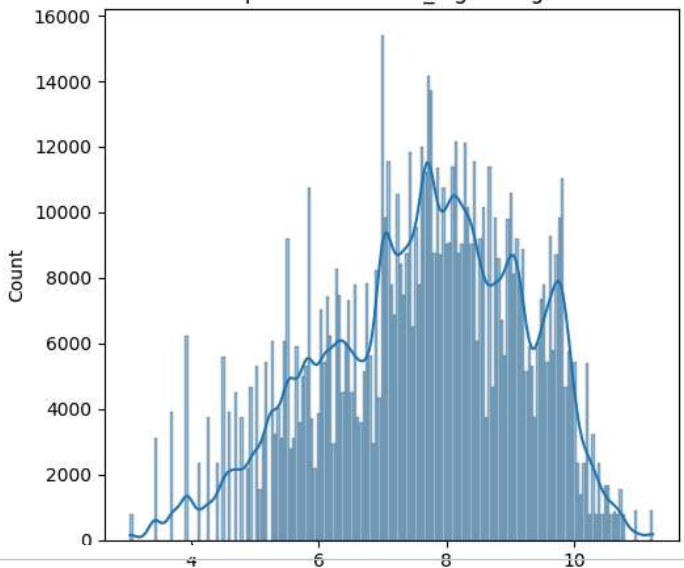
df.dtypes



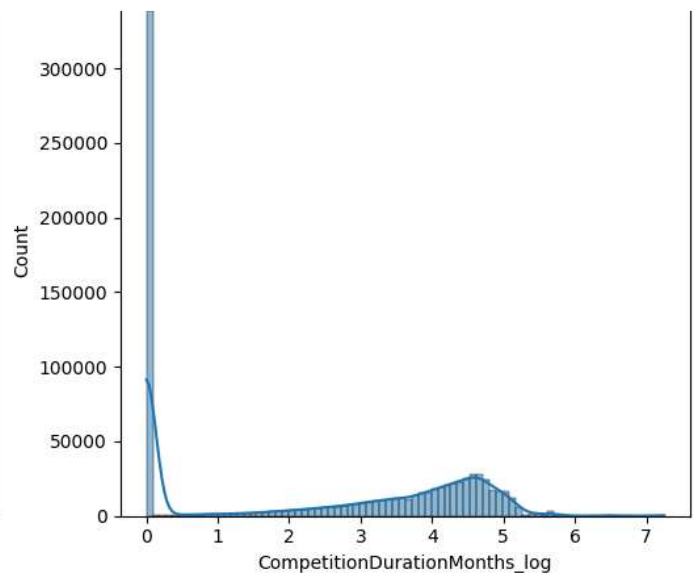
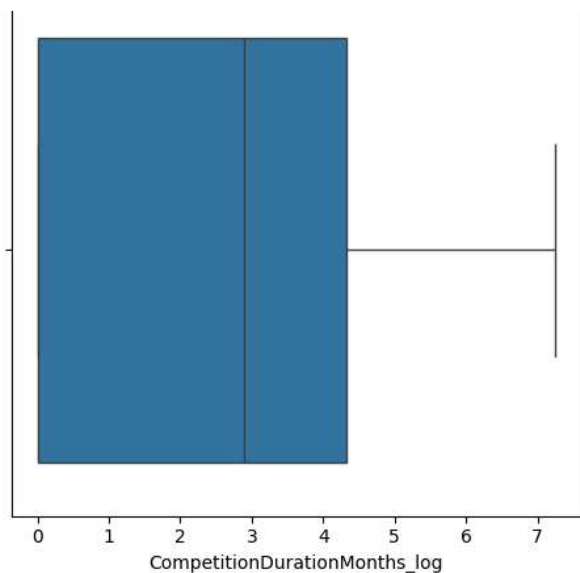
CompetitionDistance_log Boxplot



CompetitionDistance_log Histogram

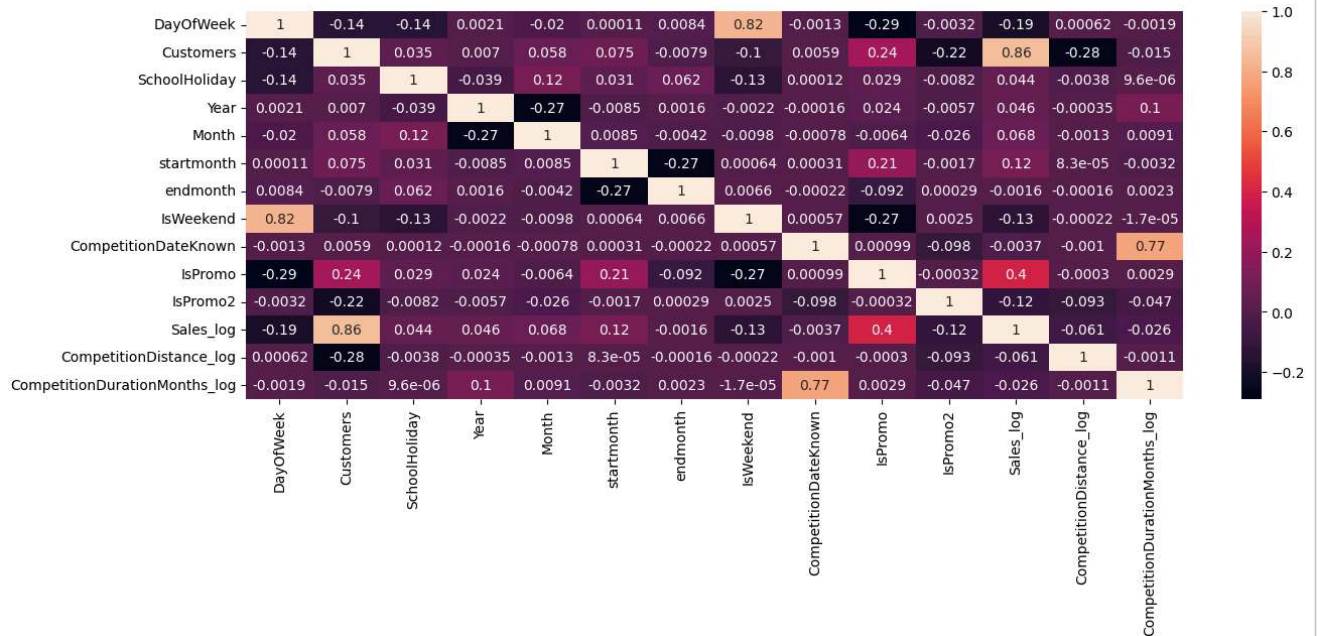


```
# Calculate the correlation between columns
data_corr=df.corr()
data_corr
```



	DayOfWeek	Customers	SchoolHoliday	Year	Month	startmonth	endmonth	IsWeekend	Competit
DayOfWeek	1.000000	-0.140932	-0.139315	0.002102	-0.019723	0.000113	0.008372	0.818721	
Customers	-0.140932	1.000000	0.035319	0.007039	0.058490	0.074999	-0.007854	-0.102300	
SchoolHoliday	-0.139315	0.035319	1.000000	-0.038679	0.118485	0.031286	0.061613	-0.128533	
Year	0.002102	0.007039	-0.038679	1.000000	-0.268819	-0.008544	0.001618	-0.002237	
Month	-0.019723	0.058490	0.118485	-0.268819	1.000000	0.008480	-0.004220	-0.009758	
startmonth	0.000113	0.074999	0.031286	-0.008544	0.008480	1.000000	-0.274589	0.000637	
endmonth	0.008372	-0.007854	0.061613	0.001618	-0.004220	-0.274589	1.000000	0.006591	
IsWeekend	0.818721	-0.102300	-0.128533	-0.002237	-0.009758	0.000637	0.006591	1.000000	
CompetitionDateKnown	-0.001309	0.005922	0.000123	-0.000160	-0.000776	0.000311	-0.000216	0.000567	
IsPromo	-0.289265	0.243761	0.028964	0.023994	-0.006408	0.206089	-0.091756	-0.272741	
IsPromo2	-0.003241	-0.219785	-0.008245	-0.005702	-0.025816	-0.001748	0.000285	0.002504	
Sales_log	-0.192943	0.855411	0.043501	0.046158	0.068450	0.115804	-0.001594	-0.126588	
CompetitionDistance_log	0.000620	-0.276570	-0.003844	-0.000354	-0.001256	0.000083	-0.000159	-0.000220	
CompetitionDurationMonths_log	-0.001922	-0.015020	0.000010	0.104932	0.009094	-0.003171	0.002258	-0.000017	

```
# Draw heatmap to show the corrlletion between columns
plt.figure(figsize=(15,5))
sns.heatmap(data_corr , annot= True)
plt.show()
```



```
# Selected feature which has good affect on the target and remove others.
selected_features = [
    "customers",
    "IsPromo",
    "DayOfWeek",
    "Month",
    "SchoolHoliday",
    "CompetitionDurationMonths_log",
    "Year",
    "IsPromo2"
]
```

```
df = df.drop(columns=[
    "IsWeekend",
    "CompetitionDateKnown",
    "CompetitionDistance_log",
    "endmonth",
    "startmonth"
])
```

7. Train_Test split

```
# Divide the two into feature and target
# x => features
x = df.drop(columns="Sales_log")
# y => target
y = df["Sales_log"]
```

```
# divide data into train data and test data
x_train , x_test , y_train , y_test = train_test_split(x,y , test_size=0.25 , random_state=42 , shuffle=True)
```

```
# Show the number of rows and columns in x_train, y_train .
x_train.shape, y_train.shape
```

```
((633294, 8), (633294,))
```

```
# Show the number of rows and columns in x_test, y_test.
x_test.shape, y_test.shape
```

```
((211098, 8), (211098,))
```

Modeling pipeline

1. Baseline Model
2. first Evaluation
3. Fine Tuning
4. Final Evaluation
5. Feature Importance Visualization
6. Model Deployment

1- Baseline Model

Modeling Use

1. LinearRegression
2. DecisionTreeRegressor
3. RandomForestRegressor
4. GradientBoostingRegressor
5. XGBRegressor

```
# Import Models from scikit learn
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.ensemble import GradientBoostingRegressor
from xgboost import XGBRegressor
```

```
# Creates instances (objects) of the machine learning models
# Notes => This objects created with default parameters
LR = LinearRegression()
DTR = DecisionTreeRegressor()
RFR = RandomForestRegressor()
GBR = GradientBoostingRegressor()
XGBR = XGBRegressor()
```

```
# This code trains (fits) each machine learning model using the training dataset.
# Notes => Fit means the model learns the relationship between the input features (x_train) and the target labels (y_train)
LR.fit(x_train , y_train)
DTR.fit(x_train , y_train)
RFR.fit(x_train , y_train)
GBR.fit(x_train , y_train)
XGBR.fit(x_train , y_train)
```

▼ XGBRegressor ⓘ ?

```
XGBRegressor(base_score=None, booster=None, callbacks=None,
             colsample_bylevel=None, colsample_bynode=None,
             colsample_bytree=None, device=None, early_stopping_rounds=None,
             enable_categorical=False, eval_metric=None, feature_types=None,
             feature_weights=None, gamma=None, grow_policy=None,
             importance_type=None, interaction_constraints=None,
             learning_rate=None, max_bin=None, max_cat_threshold=None,
             max_cat_to_onehot=None, max_delta_step=None, max_depth=None,
             max_leaves=None, min_child_weight=None, missing=nan,
             monotone_constraints=None, multi_strategy=None, n_estimators=None,
             n_jobs=None, num_parallel_tree=None, ...)
```

2- First Evaluation

R² Formula

$$R^2 = 1 - \frac{\sum_{i=1}^m (X_i - Y_i)^2}{\sum_{i=1}^m (\bar{Y} - Y_i)^2}$$

MAE Formula

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

RMSE Formula

$$RMSE = \sqrt{\frac{\sum (y_i - \hat{y}_i)^2}{N - P}}$$

1- R² (Coefficient of Determination) => Score measures how well the model explains the variability in the target variable. A higher R² value indicates that the model fits the data better and explains a larger proportion of the variance.

2- MAE (Mean Absolute Error) => represents the average absolute difference between the actual and predicted values. It provides a clear interpretation of the average prediction error.

3- RMSE (Root Mean Squared Error) => measures the square root of the average squared differences between predictions and actual values. It penalizes larger errors more heavily, making it sensitive to significant prediction mistakes.

```
# Import
from sklearn.metrics import r2_score, mean_squared_error, mean_absolute_error
# Evaluation on LinearRegression

y_hat_LR = LR.predict(x_test)

r2_LR = r2_score(y_test, y_hat_LR)
rmse_LR = np.sqrt(mean_squared_error(y_test, y_hat_LR))
mae_LR = mean_absolute_error(y_test, y_hat_LR)
```

```
# Evaluation on DecisionTreeRegressor

y_hat_DTR = DTR.predict(x_test)

r2_DTR = r2_score(y_test, y_hat_DTR)
rmse_DTR = np.sqrt(mean_squared_error(y_test, y_hat_DTR))
mae_DTR = mean_absolute_error(y_test, y_hat_DTR)
```

```
# Evaluation on RandomForestRegressor

y_hat_RFR = RFR.predict(x_test)

r2_RFR = r2_score(y_test, y_hat_RFR)
rmse_RFR = np.sqrt(mean_squared_error(y_test, y_hat_RFR))
mae_RFR = mean_absolute_error(y_test, y_hat_RFR)
```

```
# Evaluation on GradientBoostingRegressor
```

```

y_hat_GBR = GBR.predict(x_test)

r2_GBR = r2_score(y_test, y_hat_GBR)
rmse_GBR = np.sqrt(mean_squared_error(y_test, y_hat_GBR))
mae_GBR = mean_absolute_error(y_test, y_hat_GBR)

```

```
# Evalution on XGBRegressor
```

```

y_hat_XGB = XGBR.predict(x_test)

r2_XGB = r2_score(y_test, y_hat_XGB)
rmse_XGB = np.sqrt(mean_squared_error(y_test, y_hat_XGB))
mae_XGB = mean_absolute_error(y_test, y_hat_XGB)

```

```
# Compare Between five models
```

```

results = pd.DataFrame({
    "Model": ["LinearRegression", "DecisionTree", "RandomForest", "GradientBoosting", "XGBoost"],
    "R2": [r2_LR, r2_DTR, r2_RFR, r2_GBR, r2_XGB],
    "RMSE": [rmse_LR, rmse_DTR, rmse_RFR, rmse_GBR, rmse_XGB],
    "MAE": [mae_LR, mae_DTR, mae_RFR, mae_GBR, mae_XGB]
})

print(results)

```

	Model	R2	RMSE	MAE
0	LinearRegression	0.777026	0.203312	0.159744
1	DecisionTree	0.668077	0.248059	0.187428
2	RandomForest	0.808484	0.188425	0.145880
3	GradientBoosting	0.805631	0.189824	0.150480
4	XGBoost	0.808493	0.188421	0.143393

Best Model

1. XGBoost
2. GradientBoosting
3. RandomForest

3- Fine Tuning

```

# auto fine tuning (grid search)
# import
from sklearn.model_selection import GridSearchCV

```

```

# # 1- XGBRegressor

# # Object
# XGBoost2 = XGBRegressor()
# # Parameters
# params = {
#     "learning_rate": [0.1, 0.01],
#     "max_depth": [5, 7],
#     "gamma": [0, 0.1, 0.3],
#     "min_child_weight": [1, 3, 5],
#     "n_estimators": [100, 200]
# }

# # GridSearchCV object
# clf = GridSearchCV(XGBoost2, params, cv=10)
# # fit
# clf.fit(x_train, y_train)

```

```

# # Display the best parameter came out from grid search of XGBRegressor
# clf.best_params_

```

```
XGBoost2 = XGBRegressor(gamma=0, learning_rate=.1, max_depth=7, min_child_weight=1, n_estimators=200)
```

```
XGBoost2.fit(x_train, y_train)
```

▼ XGBRegressor ⓘ ?

```
XGBRegressor(base_score=None, booster=None, callbacks=None,
              colsample_bylevel=None, colsample_bynode=None,
              colsample_bytree=None, device=None, early_stopping_rounds=None,
              enable_categorical=False, eval_metric=None, feature_types=None,
              feature_weights=None, gamma=0, grow_policy=None,
              importance_type=None, interaction_constraints=None,
              learning_rate=0.1, max_bin=None, max_cat_threshold=None,
              max_cat_to_onehot=None, max_delta_step=None, max_depth=7,
              max_leaves=None, min_child_weight=1, missing=nan,
              monotone_constraints=None, multi_strategy=None, n_estimators=200,
              n_jobs=None, num_parallel_tree=None, ...)
```

```
# # 2- GradientBoostingClassifier
```

```
# # Object
# GBoost2 = GradientBoostingRegressor()
# # parameters
# parms = {
#     'learning_rate': [0.1, 0.05],
#     'n_estimators': [100, 200],
#     'max_depth': [3,5],
# }
```

```
# # GridSearchCV object
# clf = GridSearchCV(GBoost2, parms, cv=5)
# # fit
# clf.fit(x_train, y_train)
```

```
# # Display the best parameter came out from grid search of GradientBoostingClassifier
# clf.best_params_
```

```
GBoost2 = GradientBoostingRegressor(learning_rate=.1 ,max_depth=5 ,n_estimators=200)
```

```
GBoost2.fit(x_train, y_train)
```

▼ GradientBoostingRegressor ⓘ ?

```
GradientBoostingRegressor(max_depth=5, n_estimators=200)
```

```
# # fine tuning
# # auto fine tuning (grid search)
```

```
# # 3- RandomForestClassifier
```

```
# # object
# RFR2=RandomForestRegressor()
# # parameter
# parms = {
#     'criterion': ['squared_error'],
#     'max_depth': [5,10],
#     'min_samples_split': [2,5],
#     'min_samples_leaf': [1,2]
# }
# # GridSearchCV object
# clf = GridSearchCV(RFR2, parms, cv=5)
# # fit
# clf.fit(x_train, y_train)
```

```
# # Display the best parameter came out from grid search of GradientBoostingClassifier
# clf.best_params_
```

```
RFR2 = RandomForestRegressor(criterion="squared_error" ,max_depth=10 ,min_samples_leaf=2 ,min_samples_split=5)
```

```
RFR2.fit(x_train, y_train)
```

RandomForestRegressor

```
RandomForestRegressor(max_depth=10, min_samples_leaf=2, min_samples_split=5)
```

4- Final Evaluation

```
# Code to display and compare between three models

models = {
    "RandomForest": RFR2,
    "GradientBoosting": GBoost2,
    "XGBoost": XGBoost2
}

results = []

for name, model in models.items():

    y_pred = model.predict(x_test)

    r2 = r2_score(y_test, y_pred)
    rmse = np.sqrt(mean_squared_error(y_test, y_pred))
    mae = mean_absolute_error(y_test, y_pred)

    results.append({
        "Model": name,
        "R2": r2,
        "RMSE": rmse,
        "MAE": mae
    })

results = pd.DataFrame(results)

print(results)
```

	Model	R2	RMSE	MAE
0	RandomForest	0.810475	0.187443	0.148346
1	GradientBoosting	0.817411	0.183981	0.145245
2	XGBoost	0.812652	0.186363	0.141467

Final Model Result :

After evaluating the performance of Random Forest, Gradient Boosting, and XGBoost using R^2 , RMSE, and MAE metrics, GradientBoostingRegressor achieved the best overall results. It obtained the highest R^2 score (0.817412 ~ = 82%) and the lowest RMSE (0.183981 ~ = 2%) among the tested models. Although XGBoost achieved a slightly lower MAE, the difference between the models was minimal. Therefore, GradientBoostingRegressor was selected as the final model for deployment and integration into the Streamlit application.

Note => The range of target from (0) to (10.68)

5- Feature Importance Visualization

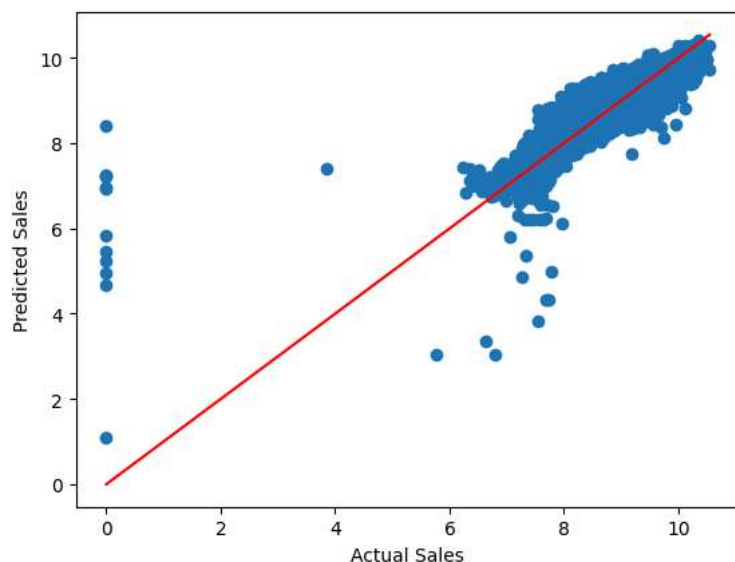
```
# Show the Relationship between Actual Sales and Predicted Sales in digram

plt.scatter(y_test, y_pred)

plt.plot([y_test.min(), y_test.max()],
         [y_test.min(), y_test.max()],
         color="red")

plt.xlabel("Actual Sales")
plt.ylabel("Predicted Sales")
```

Text(0, 0.5, 'Predicted Sales')



The scatter plot shows the relationship between actual and predicted sales. Most points lie close to the red diagonal line, which represents perfect prediction. This indicates that the model is able to capture the overall trend of the sales data and provide reasonably accurate predictions.

Note => there are some points far from the red line, possible the store is close or unusual sales or outliers and there is usual in any model.

```
# Important feature affect on the model
importance = GBoost2.feature_importances_

# Convert to dataframe
feature_importance = pd.DataFrame({
    "Feature": selected_features,
    "Importance": importance
})

# Sort the feature
feature_importance = feature_importance.sort_values(
    by="Importance",
    ascending=False
)

print(feature_importance)
```

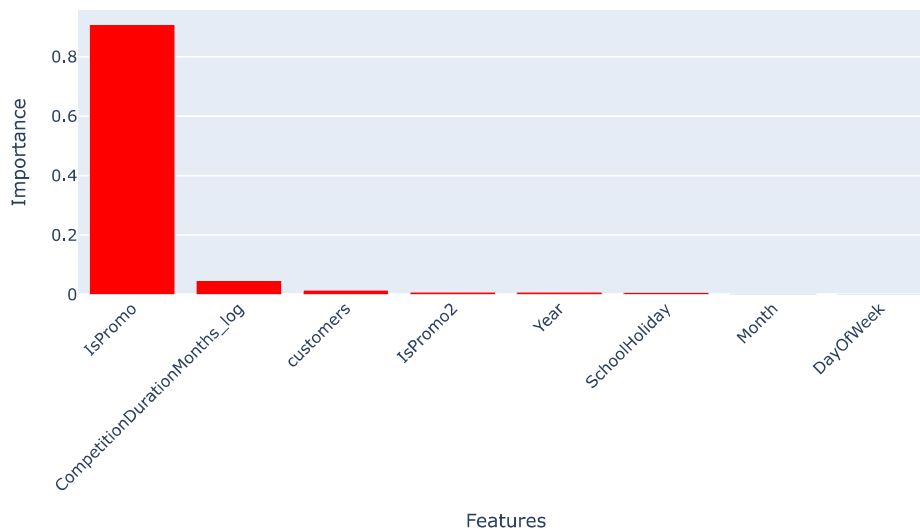
	Feature	Importance
1	IsPromo	0.908392
5	CompetitionDurationMonths_log	0.047724
0	customers	0.014647
7	IsPromo2	0.008957
6	Year	0.008802
4	SchoolHoliday	0.007681
3	Month	0.003050
2	DayOfWeek	0.000746

```
# Display the important feature which affect on the model in barplot
fig = px.bar(
    feature_importance,
    x="Feature",
    y="Importance",
    title="Feature Importance in Gradient Boosting Model",
    width=800,
    height=500,
    color_discrete_sequence=["red"]
)

fig.update_layout(
    xaxis_title="Features",
    yaxis_title="Importance",
    xaxis_tickangle=-45
)
```

```
fig.show()
```

Feature Importance in Gradient Boosting Model



```
df['Customers'].describe()
```

```

      Customers
count  844392.000000
mean      6.535078
std       0.442544
min       0.000000
25%      6.253829
50%      6.517671
75%      6.795706
max      8.907748

dtype: float64

```

5- Model Deployment

```

# 1- Divide the features which enter to the model to categorical_features and numerical_features
categorical_features = [
    "DayOfWeek",
    "SchoolHoliday",
    "IsPromo",
    "IsPromo2"
]
numerical_features = [
    "Customers",
    "Year",
    "Month",
    "CompetitionDurationMonths_log"
]

```

```

# Import
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline

```

```

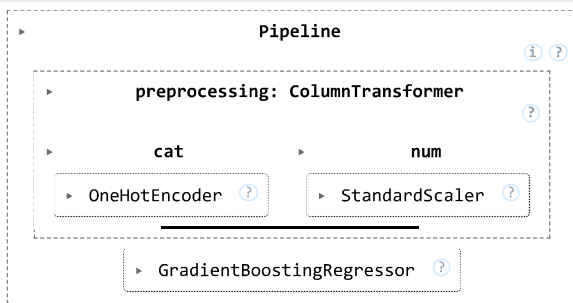
# Create Preprocessor
preprocessor = ColumnTransformer(
    transformers=[

```

```

        ("cat", OneHotEncoder(handle_unknown="ignore"), categorical_features),
        ("num", StandardScaler(), numerical_features)
    ]
)
# Create pipeline
pipeline = Pipeline([
    ("preprocessing", preprocessor),
    ("model", GBoost2)
])
# train pipeline
pipeline.fit(x_train, y_train)

```



```
# Code to check that pipeline train good and compare the result before and after pipeline
```

```

# test
pred = pipeline.predict(x_test)

mae = mean_absolute_error(y_test, pred)
rmse = np.sqrt(mean_squared_error(y_test, pred))
r2 = r2_score(y_test, pred)

print("MAE:", mae)
print("RMSE:", rmse)
print("R2:", r2)

```

```

MAE: 0.1453447323654313
RMSE: 0.18401448940211765
R2: 0.8173451150982713

```

```

# Import
import joblib
# Save pipeline (Preprocessing + model)
joblib.dump(pipeline, "demand_forecasting_model.pkl")

```

```
['demand_forecasting_model.pkl']
```

```

# from google.colab import files
# files.download("demand_forecasting_model.pkl")

```

Display the model in GUI using Streamlit

```

# %%writefile app.py
# import streamlit as st
# import pandas as pd
# import joblib
# import numpy as np

# # Load model
# model = joblib.load("demand_forecasting_model.pkl")

# # Page config
# st.set_page_config(page_title="Store Sales Prediction", layout="wide")

# # Title
# st.title("📊 Store Sales Prediction Dashboard")
# st.write("Predict store sales based on store conditions.")

# # Layout
# col1, col2 = st.columns(2)

```